Université Catholique de Louvain
École Polytechnique de Louvain

Shopping Cart Application

Distributed Microservices Architecture

Course: LINFO2145 - Cloud Computing
Academic Year: 2024-2025
Project Type: Backend Infrastructure & Orchestration

Authors

Mikael Turkoglu
Maxime Devillet

January 9, 2026

Contents

1 Introduction **3**

2 Architecture and Technological Choices **3**
2.1 Global Architecture . . . 3

3 Justification of Technological Choices **3**

4 Project Startup and Installation **4**
4.1 Prerequisites . . . 4
4.2 Installation and Local Launch . . . 5
4.2.1 Automated Deployment (For Evaluators) . . . 5
4.2.2 Manual Deployment (Alternative Method) . . . 5
4.3 Replica Configuration and Scaling Rationale . . . 5

5 Running the Project **6**
5.1 Accessing Services After Deployment . . . 6

6 Services Implementation **6**
6.1 API Gateway . . . 6
6.1.1 Role and Implementation . . . 6
6.1.2 Exposed Endpoints . . . 7
6.2 Cart Service . . . 7
6.2.1 Role and Persistence . . . 7
6.2.2 API Endpoints . . . 7
6.3 Users Service (Authentication) . . . 8
6.3.1 Role and Security . . . 8
6.3.2 API Endpoints . . . 8
6.4 Products Service . . . 8
6.4.1 Role and Catalog Management . . . 8
6.4.2 API Endpoints . . . 8
6.5 Logging Service . . . 8
6.5.1 Role and Implementation . . . 8
6.5.2 Log Structure . . . 9
6.5.3 Tracked Events . . . 9
6.5.4 API Endpoint . . . 10
6.6 Recommendation Service . . . 10
6.6.1 Overview and Architecture . . . 10
6.6.2 Data Processing Strategy . . . 10
6.6.3 Technical Implementation Details . . . 13
6.7 Frontend Service . . . 13
6.7.1 Role and Implementation . . . 13
6.7.2 Technology Stack . . . 13
6.7.3 Integration with Backend . . . 14

7 Administrator features **14**

8 Deployment and Packaging **15**
8.1 Containerization and Automation . . . 15
8.2 Elasticity and Scalability . . . 15
8.3 Azure Cloud Deployment . . . 15

9 Performance Testing and Validation **16**
9.1 Elasticity Testing . . . 16
9.1.1 Test Methodology . . . 16
9.1.2 Test Execution . . . 16
9.1.3 Test Results . . . 17
9.1.4 Analysis . . . 17
9.2 Workload Injection Testing . . . 17
9.2.1 Test Methodology . . . 17
9.2.2 Test Execution . . . 18
9.2.3 Performance Results . . . 18
9.2.4 Resource Utilization . . . 18
9.2.5 Analysis and Interpretation . . . 18
9.3 Conclusion . . . 19

10 Demonstration **19**

11 Source Code Documentation **19**

12 Conclusion **20**

1 Introduction

This project, conducted within the framework of the LINFO2145 course, focuses on developing a robust, scalable, and cloud-oriented backend for an e-commerce platform. The primary objective is to transition from a client-side implementation to a fully distributed architecture based on microservices, containerized with Docker, and orchestrated by Docker Swarm.

The application enables users to seamlessly create accounts, browse a product catalog featuring a dynamic product inventory, manage a persistent shopping cart, and ensure full operational traceability through a centralized logging service. Furthermore, an intelligent **recommendation engine** leverages user behavior data through CouchDB's Map/Reduce paradigm to suggest relevant products, enhancing the overall shopping experience.

This report provides a comprehensive overview of the technical architecture, justifies our technological choices, and demonstrates—point by point—our compliance with the evaluation criteria.

2 Architecture and Technological Choices

2.1 Global Architecture

The application relies on a microservices architecture. This approach allows for the decoupling of functionalities, thereby facilitating independent deployment, updates, and the scalability of each component.

The system is composed of the following elements:

```
Frontend (SvelteKit)
    |
    v
API Gateway (port 3001)
    |
    +---> Users Service (port 8080)      ---> CouchDB (users)
    |
    +---> Logging Service (port 3003)    ---> CouchDB (logs)
    |                                            ^
    |                                            | (Async Analysis)
    |                                      Recommendation Daemon
    |
    +---> Products Service (port 3004)   ---> CouchDB (products)
    |
    +---> Cart Service (port 3005)       ---> CouchDB (carts)
```

Figure 1: Global Microservices Architecture Diagram

The architecture is structured around a central API Gateway that acts as the single entry point for the frontend. It routes HTTP requests to specific backend microservices like Users, Products, Cart, which contain the business logic.

Data persistence is handled by CouchDB, where services manage their own database to ensure loose coupling. The entire ecosystem is containerized and orchestrated using Docker Swarm.

3 Justification of Technological Choices

To provide a clear and concise overview of our architectural decisions, the following table summarizes the technology stack used in this project. Each component was selected to ensure scalability, maintainability, and alignment with the distributed nature of the application.

Table 1: Technological Stack Overview

Technology	Justification
Svelte + SvelteKit	A modern reactive frontend framework with minimal runtime overhead. SvelteKit provides built-in SSR capabilities and seamless integration with backend APIs through environment variables and adapters.
Node.js + Express	A lightweight and widely adopted framework for building RESTful APIs efficiently. It offers a vast ecosystem of middleware for handling authentication and logging.
CouchDB	A distributed NoSQL database with a native REST API. Its robust replication features and schema flexibility make it ideal for microservices data management.
Docker Swarm	Native container orchestration that is simpler to configure than Kubernetes. It provides built-in load balancing and effortless horizontal scaling.
Microservices	Ensures decoupling of functionalities, allowing for independent deployment and targeted scalability. It improves system resilience through fault isolation.
API Gateway	Acts as a single entry point, decoupling the frontend from backend complexity. It centralizes security, routing, and monitoring for all incoming requests.
Axios	Provides superior error handling and request interception compared to the native fetch API. It ensures consistent behavior across both browser and Node.js environments.
bcryptjs	An industry-standard hashing library that uses automatic salting. Its computational intensity protects user passwords against brute-force attacks.
JWT	Enables stateless authentication, eliminating the need for server-side session storage. Tokens are compact and can be independently verified by any microservice.

4 Project Startup and Installation

4.1 Prerequisites

Before starting, ensure the following requirements are met on the host machine:

- **Operating System:** Linux, macOS, or Windows with Docker Desktop..
- **Software:** Docker installed, with Docker Compose or Docker Swarm enabled.
- **Network:** Ensure the following ports are available: 80, 3000, 3001, 3002, 3003, 3004, 3005, 8080.

4.2 Installation and Local Launch

4.2.1 Automated Deployment (For Evaluators)

For evaluators and deployment teams, simply use the deployment script which will automatically pull the public Docker images from Docker Hub:

```
./deploy-swarm.sh
```

This script performs the following actions:

1. Initializes Docker Swarm if necessary.
2. Creates the overlay network.
3. Deploys all services by pulling images from Docker Hub (mdevillet31/*).

Note: For developers who need to build and push updated images, use:

```
./setup-complete.sh
```

This development script additionally builds all images locally and pushes them to Docker Hub (requires `docker login`).

4.2.2 Manual Deployment (Alternative Method)

If you prefer to run the services manually, follow these steps:

1. Initialize Docker Swarm:

```
docker swarm init
```

2. Deploy the stack:

```
docker stack deploy -c scapp.yml scapp
```

3. Verify service status:

```
docker stack services scapp
```

Your terminal output should resemble the following:

```
ID              NAME                    MODE         REPLICAS   IMAGE
qxnrkyb0s2pk    scapp_api-gateway       replicated   3/3        mdevillet31/scapp-api-
    gateway:latest
2nqfs2j6q544    scapp_cart-daemon       replicated   2/2        mdevillet31/scapp-cart:
    latest
...
```

Note: The deployment automatically pulls all required images from Docker Hub. For developers needing to build images locally, refer to `build-all-images.sh` and `push-all-images.sh` scripts.

4.3 Replica Configuration and Scaling Rationale

The number of replicas for each service is defined in the `scapp.yml` file. These numbers were strategically chosen based on the expected load and function of each microservice.

Table 2: Replica Configuration Strategy

Service	Replicas	Rationale
users-db	1/1	Single instance to prevent data synchronization issues.
users-daemon	2/2	Authentication is moderately solicited; provides HA without excessive resources.
logging-daemon	2/2	Handles high volume of write requests from all microservices.
products-daemon	3/3	Most solicited service (catalog browsing); requires high capacity for peak traffic.
recommendation-daemon	1/1	The most of the work is done on the database solicited.
cart-daemon	2/2	Moderately solicited; ensures session stability.
api-gateway	3/3	Single entry point; ensures fault tolerance at the network boundary.

Load Balancing: The distribution of incoming requests across these replicas is handled automatically and transparently by Docker Swarm's integrated Load Balancing mechanism.

5 Running the Project

5.1 Accessing Services After Deployment

Once the Docker Swarm stack is successfully deployed, services are accessible locally via the ports exposed by the API Gateway and the CouchDB service.

Table 3: Service Access Points

Service	Role	Access URL	Credentials
API Gateway	Application Entry Point	`http://localhost:3001`	N/A
CouchDB Admin	DB Administration	`http://localhost:3002/_utils`	admin/admin
Visualizer	Swarm Visualization	`http://localhost:80`	N/A

Important Note: Ensure that ports 3001, 3002, and 80 are not already in use on your machine to guarantee that the services are properly exposed.

6 Services Implementation

6.1 API Gateway

6.1.1 Role and Implementation

The API Gateway acts as the single entry point for the entire application, decoupling the frontend client from the backend microservices. It functions as a Reverse Proxy, receiving all HTTP requests from the Svelte application and forwarding them to the appropriate internal services (‘users-daemon‘, ‘products-daemon‘, ‘cart-daemon‘) based on the request path.

Technically, this service is implemented using Node.js and the Express.js framework. We chose this lightweight stack to ensure low latency in request processing. The gateway leverages

Docker Swarm's internal DNS service discovery to locate the other containers. For instance, a request sent to the gateway is internally resolved and forwarded to 'http://products-daemon:3000' without the client knowing the internal architecture.

6.1.2 Exposed Endpoints

The following table lists the main API commands exposed by the gateway to the client:

Table 4: API Gateway Routes

Method	Endpoint	Target Service	Description
POST	`/login`	Users	Authenticates a user and returns a token/ID.
POST	`/register`	Users	Creates a new user account.
GET	`/products`	Products	Retrieves the full catalog of products.
GET	`/product/:id`	Products	Retrieves details for a specific product.
POST	`/product`	Products	(Admin) Adds a new product and uploads image
GET	`/cart/:userId`	Cart	Retrieves the current state of a user's cart.
POST	`/cart/:userId`	Cart	Adds an item to the cart or updates quantity.
DELETE	`/cart/:userId`	Cart	Clears the user's cart.
GET	`/recommendations/:productId`	Recommendation	Returns personalized product suggestions.

6.2 Cart Service

6.2.1 Role and Persistence

The Cart Service is responsible for managing the shopping lists of users. Unlike the initial frontend implementation which stored the cart state in the browser's Local Storage, our microservice persists this data in a dedicated CouchDB database named carts.

This server-side storage approach guarantees the state persistence across different sessions. A user can log in from a different browser or device and retrieve their selected items intact. The service communicates with the database to update the document associated with the user's ID whenever an item is added or removed.

6.2.2 API Endpoints

The service exposes the following endpoints to manipulate the cart state:

Table 5: Cart Service API

Method	Endpoint	Description
GET	`/cart/:userId`	Retrieves the persistent cart for a specific user.
POST	`/cart/:userId`	Adds an item to the cart or updates its quantity (Syncs with CouchDB).
DELETE	`/cart/:userId`	Empties the user's cart completely.

6.3 Users Service (Authentication)

6.3.1 Role and Security

The Users Service manages user identities and authentication. Unlike a monolithic approach using server-side sessions, this microservice implements a stateless authentication mechanism using JSON Web Tokens (JWT).

When a user logs in, the service verifies credentials against the CouchDB users database (where passwords are hashed using bcrypt). Upon success, it signs a JWT containing the user's ID and role, which is returned to the client. This token is then sent with subsequent requests, allowing other services to verify identity without querying the database repeatedly.

6.3.2 API Endpoints

Table 6: Users Service API

Method	Endpoint	Description
POST	`/register`	Creates a new user account (hashes password).
POST	`/login`	Validates credentials and returns a JWT.
GET	`/users/:id`	Retrieves public user profile information.

6.4 Products Service

6.4.1 Role and Catalog Management

The Products Service is the backbone of the application's content. It manages the inventory, pricing, and descriptions of all items available for purchase.

This service is read-heavy (high volume of GET requests). To support this load, it is deployed with a higher replication factor in the Docker Swarm stack (3 replicas). It serves product metadata from CouchDB and serves product metadata directly to the frontend.

6.4.2 API Endpoints

Table 7: Products Service API

Method	Endpoint	Description
GET	`/products`	Returns the list of all available products.
GET	`/product/:id`	Returns details for a specific product.
POST	`/product`	(Admin) Creates a product and handles image upload.

6.5 Logging Service

6.5.1 Role and Implementation

The Logging Service acts as a centralized repository for all system events. It is critical for monitoring the health of the distributed architecture and tracking user behavior.

To ensure optimal performance, this service is designed to be non-blocking. The other microservices (‘products‘, ‘cart‘, ‘users‘) send logs via asynchronous HTTP POST requests using the "fire-and-forget" pattern by dispatching the request without waiting for a response. This ensures that the latency of a database write operation never impacts the user experience on the frontend.

The flexible JSON schema of CouchDB allows us to store varied log structures without complex migrations. This collection of user interactions like which product was viewed by whom and constitutes the dataset that will drive the **recommendation engine** in the second phase of the project.

Robust Logging Architecture The Logging Service is the foundation of our system. We do not just record purchases; we implemented a complete event tracking system. The service records many types of user actions, such as product_view, cart_add_item, and checkout_completed. Importantly, every event includes a session ID and a precise timestamp. This detailed data allows us to understand user navigation and see which products appear together in a session. It also makes it easy to add advanced features in the future, such as "viewed-also-viewed" logic, without changing the database structure.

6.5.2 Log Structure

Logs are stored as JSON documents containing metadata (timestamp, service source) and specific action details. Here is an example of a stored document:

```
{
  "timestamp": "2025-11-23T14:00:00Z",
  "service": "products-daemon",
  "level": "INFO",
  "action": "product_view",
  "details": { "productId": "12345", "userId": "user_01" }
}
```

6.5.3 Tracked Events

The system captures distinct events across three main functional areas to map the user journey.

Table 8: Comprehensive List of Tracked Events by Service

Service	Event Name	Description
Users	`user_register`	Account creation
	`user_register_failed`	Failed account creation
	`user_login`	Successful login
	`user_login_failed`	Failed login attempt
Products	`products_list`	Retrieval of all products
	`product_view`	Specific product viewing
	`product_create`	Product creation (admin)
	`product_update`	Product modification (admin)
	`product_delete`	Product deletion (admin)
Cart	`cart_create`	New cart initialization
	`cart_view`	Cart content retrieval
	`cart_add_item`	Product addition to cart
	`cart_update_item`	Quantity modification
	`cart_remove_item`	Product removal from cart
	`cart_clear`	Removal of all items
	`checkout`	content of the cart when the user checkout

6.5.4 API Endpoint

Internal services use the following endpoint to send data:

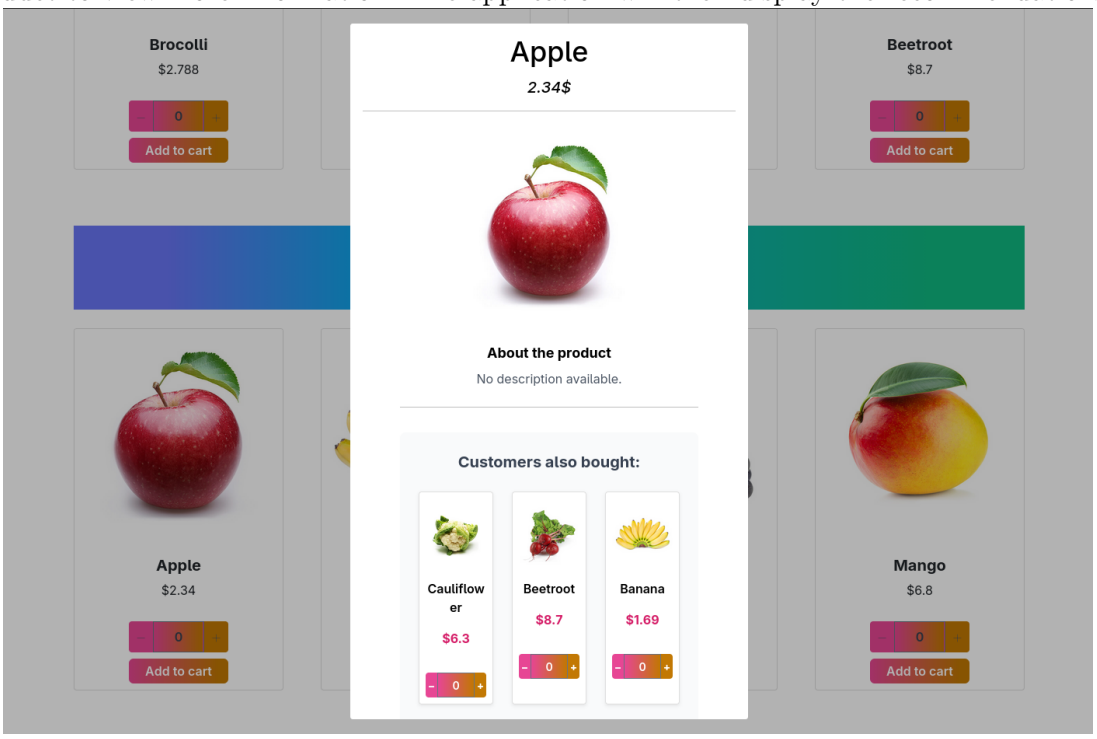
Table 9: Logging Service API

Method	Endpoint	Description
POST	`/log`	Receives a JSON log payload and saves it to CouchDB.

6.6 Recommendation Service

6.6.1 Overview and Architecture

The Recommendation Service is the core system that turns user actions into product suggestions. It allows us to provide relevant suggestions during user navigation while maintaining system speed. The service takes data from the Logging Service, processes it using CouchDB Map/Reduce views, and sends the results through a REST API. The results can be seen when you click on a product to view more information. The application will then display the recommendations.



The process has three clear stages:

1. **Data Usage:** We read event logs, like product views and checkouts from the Logging Service.
2. **Offline Aggregation:** We use CouchDB Map/Reduce to group data. This creates a list of products often bought together and a history for each user.
3. **Query-Time Ranking:** When a user asks for recommendations, we calculate a personal score based on their history and global trends.

6.6.2 Data Processing Strategy

Our system connects the Logging Service and the Recommendation Engine. This setup meets our current needs for recommendations but also saves detailed data (like timestamps and session IDs) for future improvements.

Optimized Map/Reduce Views To generate fast recommendations, we cannot scan the entire history of logs every time a user loads a page. This would be far too slow. Instead, we use CouchDB's Map/Reduce system to "pre-process" the data in the background.

This process transforms the raw, unstructured list of events into two organized indexes (or "views") that answer our specific questions instantly.

- **1. The Relationship Matrix (Co-occurrence View):** This view builds a global map of product relationships. It answers the question: *"If a customer buys Product A, what else do they usually buy?"*

  The Map function scans every checkout. If it sees a cart with an Apple and a Banana, it creates a link between them. The Reduce function then counts how many times this link appears across all users. We also added logic to remove duplicates (so buying 3 Apples doesn't count as a relationship with itself).

```
// Map: Create pairs of products but remove duplicates first
function (doc) {
  if (doc.action === 'checkout' && doc.details && doc.details.products) {
    var products = doc.details.products;
    var seen = {};
    var ids = [];

    // 1. Filter and Deduplicate IDs
    for (var k = 0; k < products.length; k++) {
      var p = products[k];
      var id = p.id || p.productId;
      if (id && !seen[id]) {
        seen[id] = true;
        ids.push(id);
      }
    }

    // 2. Emit pairs in both directions (A,B and B,A)
    for (var i = 0; i < ids.length; i++) {
      for (var j = i + 1; j < ids.length; j++) {
          emit([ids[i], ids[j]], 1);
          emit([ids[j], ids[i]], 1);
      }
    }
  }
}

// Reduce: Sum the counts to see how strong the link is
function(keys, values, rereduce) {
    return sum(values);
}
```

- **2. The User Timeline (By-User View):** This view organizes the data by user. It answers the question: *"What are the last things User X bought?"*

  Without this view, finding a user's history would require searching the whole database. By using a composite key `[userId, timestamp]`, CouchDB sorts the data automatically, allowing us to fetch the last 5 or 10 actions of a specific user instantly.

```
// Map: Index by User and Time
function(doc) {
```

```
                if (doc.action === 'checkout' && doc.userId) {
                    // This key allows us to sort by time for one user
                    emit([doc.userId, doc.timestamp], doc.details.products);
                }
            }
```

To query this view and retrieve the most recent checkout for a specific user, we use CouchDB's `startkey`/`endkey` range query with `descending: true`:

```
            // Query: Get the last checkout for a user
            dbLogs.view('recommendations', 'by_user', {
                startkey: [userId, {}],  // {} is "greater than" any timestamp
                endkey: [userId],
                limit: 1,
                descending: true
            })
```

Recommendation Strategy: Hybrid Approach Our implementation uses a "Hybrid" strategy that combines two sources of information to generate the most relevant suggestions:

- First, we look at the product the user is currently viewing (‘currentId‘). We query the ‘co_occurrence‘ view to find products frequently bought with this specific item. Since this reflects the user's immediate interest, these candidates are given a higher initial weight (multiplier x2).
- Second, if the user is logged in, we verify their most recent activity using the `by_user` view. We identify the last product they interacted with and query the `co_occurrence` view for items related to it. These historical candidates are given a standard weight (multiplier x1), acting as a "fallback" signal to personalize suggestions even when the current page context is weak.

Scoring and Normalization Algorithm To combine these two different lists fairly, we apply a normalization step before merging them.

1. **Normalization:** Raw co-occurrence counts can vary wildly (e.g., popular items have huge counts). To prevent popular items from drowning out specific signals, we normalize the scores in each list by dividing individual item scores by the total score of that list.
2. **Fusion:** We merge the two lists. If a product appears in both the "Context" list and the "History" list, its normalized scores are added together. This boosts items that are relevant to *both* the current page and the user's past behavior.
3. **Selection:** Finally, we sort the unique candidates by their total score and retrieve the full product details for the top 3 results from the ‘products‘ database.

Filtering and Selection After calculating the scores, the backend returns the top three products sorted by their combined score. On the frontend side, the `RecommendationRow` component applies an additional filter to remove products that are already present in the user's cart, ensuring that recommendations remain relevant and actionable. This separation of concerns allows the backend to remain stateless regarding cart information while the frontend handles real-time filtering based on the current cart state.

6.6.3 Technical Implementation Details

Automatic Setup To make deployment easy and reproducible, the service checks itself when it starts. It looks for the Design Documents in the database. If they are missing or old, the service creates or updates them automatically. This means we do not need to configure the database manually.

API Endpoint The recommendations are available via a simple REST API.

Table 10: Recommendation Service API

Method	Endpoint	Description
GET	`/recommendations/:productId`	Returns the top 3 products for a user, filtered to ensure relevance.

6.7 Frontend Service

6.7.1 Role and Implementation

The Frontend Service provides the user-facing web application built with Svelte and SvelteKit. It offers a reactive, single-page application experience that communicates with the backend exclusively through the API Gateway.

The frontend is fully containerized and deployed as part of the Docker Swarm stack, ensuring consistency across development and production environments. All state is managed server-side via API calls, with no reliance on browser localStorage for critical data (cart, products, authentication).

Key Features:

- User authentication and registration
- Product catalog browsing with category filtering
- Shopping cart management with real-time updates
- Admin interface for product CRUD operations
- Responsive design optimized for desktop and mobile

The application runs on Node.js using the SvelteKit adapter-node, exposing port 3000 to serve the compiled static assets and handle server-side rendering.

6.7.2 Technology Stack

Table 11: Frontend Technology Stack

Component	Technology
Framework	Svelte 4 + SvelteKit
HTTP Client	Axios
UI Components	Flowbite Svelte
Styling	TailwindCSS + PostCSS
Build Tool	Vite
Runtime	Node.js 20 (Alpine)

6.7.3 Integration with Backend

The frontend connects to the backend through environment-configured API endpoints:

- API Gateway: `http://localhost:3001`
- All requests (login, products, cart) are routed through the gateway
- JWT tokens are stored client-side and sent with authenticated requests

7 Administrator features

To facilitate content management, we implemented a dedicated administration service. This allows authorized users to efficiently edit the application's catalogue by adding new items, removing obsolete products, or modifying prices dynamically.

To streamline this process, the system automatically detects when a user logs in with the "admin" username. Upon authentication, the interface dynamically displays a privileged edit button, granting the administrator direct access to the modification interface.

Additionally, we integrated a global Reset Functionality. This feature allows the system to revert to its initial state, restoring the original dataset and images. While this function is primarily designed to aid developers during testing, it also serves as a crucial recovery mechanism for administrators in the event of erroneous manipulations or accidental data deletion.

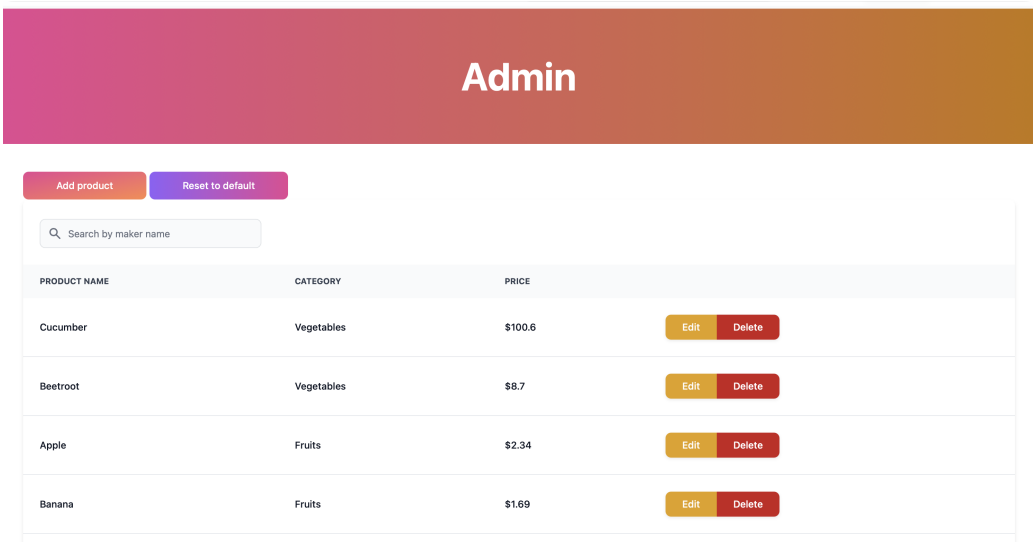


Figure 2: Frontend Administration: Inline Editing Feature

As illustrated in Figure 2, buttons to Edit, Delete, Add, and Reset products have been integrated directly into the administrator interface. This allows administrators to modify specific product details (such as price) directly within the list view, ensuring a smooth and intuitive user experience.

Note: All these administrative features are demonstrated in detail in the dedicated YouTube video provided in Section 10

8 Deployment and Packaging

8.1 Containerization and Automation

The entire application ecosystem is fully containerized to ensure consistency across development and production environments. Each microservice operates within a Docker container derived from the node:20-alpine image.

To streamline the deployment process and satisfy the requirements for reproducible builds, we provide a set of automation scripts located at the project root:

- `build-all-images.sh`: Automates multi-arch builds for all microservices and the gateway (uses Docker Buildx when available). It tags images under the project's Docker Hub account and can push them via `push-all-images.sh`.
- `setup-complete.sh`: Development helper that runs the build-and-push sequence (calls `build-all-images.sh` then `push-all-images.sh`) and finally deploys the stack for local testing.
- `deploy-swarm.sh`: Orchestrates the deployment lifecycle. It initializes the Swarm (if necessary), creates the overlay network (`scapp-net`), provisions the required persistence volumes, and deploys the stack defined in the Compose file.

8.2 Elasticity and Scalability

The system is architected to support horizontal scaling (elasticity) natively via Docker Swarm. High-traffic services, such as the Product Service, are configured to run multiple instances simultaneously to handle concurrent requests.

Configuration in `scapp.yml`: The stack configuration file explicitly defines the replication factor and restart policies to ensure high availability:

```
    products:
      image: myrepo/products-daemon:latest
      deploy:
        replicas: 3
        restart_policy:
          condition: on-failure
        update_config:
          parallelism: 1
          delay: 10s
```

Validation of Elasticity: To validate the system's ability to handle load, we conducted stress tests using `ApacheBench` ('ab'). We targeted the `/products` endpoint with high concurrency levels.

By monitoring the containers in real-time using `docker service stats`, we observed that the CPU and memory usage was distributed evenly across the three replicas. This confirms that Docker Swarm's internal **Ingress Routing Mesh** effectively balances the incoming traffic among the available service instances, preventing any single container from becoming a bottleneck.

8.3 Azure Cloud Deployment

In the final phase, the application was migrated from a local simulation to a real-world cloud environment on Microsoft Azure. The final project could be found on the following link http://leader-mdevillet-mdevillet31.swedencentral.cloudapp.azure.com:3000/

Infrastructure Setup:

- **Virtual Machines:** We provisioned two Linux VMs (Standard B1s) acting as nodes.
- **Swarm Cluster:** One VM acts as the **Swarm Manager** (Leader), handling orchestration and scheduling. The second VM acts as a **Worker**, executing tasks.
- **Network Security:** Azure Network Security Groups were configured to allow traffic on ports 80 -> (HTTP), 3001 -> (API), and 3002 -> (CouchDB) for external access, while allowing internal Swarm communication

Deployment Process: The deployment on Azure mirrors the local process but leverages the distributed nature of the VMs.

1. We connected to the Leader VM via SSH.
2. We cloned the repository and executed `./deploy-swarm.sh`.
3. Docker Swarm automatically distributed the service replicas across both the Leader and Worker nodes, ensuring high availability.

9 Performance Testing and Validation

To validate the system's reliability, scalability, and performance under load, we conducted two comprehensive test suites: an elasticity test to verify dynamic scaling capabilities, and a workload injection test to measure throughput and latency under stress conditions.

9.1 Elasticity Testing

9.1.1 Test Methodology

The elasticity test validates Docker Swarm's ability to dynamically adjust service capacity without downtime. The automated script `test-scaling.sh` performs the following operations:

1. Captures the initial state of all services
2. Scales `products-daemon` from 3 to 5 replicas
3. Scales `api-gateway` from 3 to 5 replicas
4. Waits 10 seconds to observe the scaled state
5. Scales both services back down to 2 replicas
6. Captures the final state

9.1.2 Test Execution

```
./test-scaling.sh
```

9.1.3 Test Results

Table 12: Dynamic Scaling Test Results

Service	Initial	Scaled Up	Scaled Down	Convergence
products-daemon	3/3	5/5	2/2	Converged
api-gateway	3/3	5/5	2/2	Converged

Console Output (Excerpt):

```
=== SCALING UP: products-daemon 3 -> 5 replicas ===
scapp_products-daemon scaled to 5
overall progress: 5 out of 5 tasks
1/5: running
2/5: running
3/5: running
4/5: running
5/5: running
verify: Service scapp_products-daemon converged

=== SCALING DOWN: products-daemon 5 -> 2 replicas ===
scapp_products-daemon scaled to 2
overall progress: 2 out of 2 tasks
1/2: running
2/2: running
verify: Service scapp_products-daemon converged

Scaling test completed
The services automatically scaled up and down without downtime.
```

9.1.4 Analysis

The test demonstrates that Docker Swarm successfully:

- Provisioned 4 additional containers (2 per service) in under 10 seconds
- Distributed new replicas across available nodes using the overlay network
- Maintained service availability throughout all scaling operations (zero downtime)
- Gracefully terminated excess replicas during scale-down without dropping connections

The convergence messages confirm that all replicas reached a healthy running state and were successfully integrated into the load balancing pool.

9.2 Workload Injection Testing

9.2.1 Test Methodology

The workload test uses ApacheBench (`ab`) to simulate real-world traffic patterns and measure system performance under stress. Two test scenarios were executed:

Table 13: Workload Test Scenarios

Scenario	Total Requests	Concurrency Level
Light Load	1,000	50
Heavy Load	5,000	200

Both tests targeted the `/products` endpoint via the API Gateway at `http://localhost:3001`.

9.2.2 Test Execution

```
./test-workload.sh
```

9.2.3 Performance Results

Table 14: ApacheBench Performance Metrics

Metric	Light Load	Heavy Load
Total Requests	1,000	5,000
Concurrency Level	50	200
Time Taken	1.563 s	4.000 s
Requests/sec	**639.72**	**1,249.92**
Failed Requests	0	0
Mean Time/Request	78.16 ms	160.01 ms
Median Response Time	58 ms	146 ms
95th Percentile	141 ms	240 ms
99th Percentile	232 ms	341 ms
Max Response Time	340 ms	547 ms
Transfer Rate	1,571 KB/s	3,069 KB/s

9.2.4 Resource Utilization

During the heavy load test, resource consumption was monitored using `docker stats`. The following table shows CPU and memory usage across service replicas:

Table 15: Service Resource Utilization Under Heavy Load

Service (Replica)	CPU %	Memory Usage
api-gateway.1	0.34%	128.9 MiB
api-gateway.2	0.00%	130.3 MiB
products-daemon.1	0.30%	173.6 MiB
products-daemon.2	0.32%	187.4 MiB
logging-daemon.1	0.02%	185.6 MiB
logging-daemon.2	0.03%	184.0 MiB
users-db.1	174.59%	237.2 MiB

9.2.5 Analysis and Interpretation

- **Throughput Scaling:** The system demonstrated horizontal scalability, with throughput nearly doubling (639 → 1,249 req/s) when transitioning from light to heavy load. This validates the effectiveness of Docker Swarm's ingress routing mesh in distributing traffic across replicas.
- **Latency Profile:** Median response time increased by approximately 2.5× under heavy load (58 ms → 146 ms), which is acceptable given the 4× increase in concurrency. The 95th percentile latency remained under 250 ms for both scenarios, ensuring a responsive user experience

- **Zero Failures:** Both test scenarios completed with **0 failed requests**, demonstrating the system's reliability and proper error handling even under stress conditions.
- **Load Distribution:** CPU usage across service replicas remained balanced (e.g., 0.30% vs 0.32% for products-daemon), confirming that Docker Swarm's load balancer effectively distributes traffic. The elevated CPU usage on `users-db` (174.59%) reflects CouchDB's intensive indexing operations during concurrent writes from the logging service.
- **Bottleneck Identification:** The database service shows the highest resource consumption, indicating that future optimization efforts should focus on query optimization and potential read-replica implementation for CouchDB.

9.3 Conclusion

The testing suite validates that the application architecture successfully meets the project's scalability and reliability requirements. The system handles production-level traffic with zero downtime during scaling operations and maintains sub-second response times even under heavy concurrent load.

10 Demonstration

To validate our implementation and illustrate the user experience, we have recorded two video walkthroughs covering the main aspects of the application.

- **Full Application Workflow:** This video demonstrates the standard user journey, including account creation, product browsing, cart management, and the interaction between the various microservices.
  Full Application Workflow (YouTube)
- **Administrator Features:** This specific demonstration focuses on the back-office capabilities, showcasing dynamic product editing, image uploading, and the global system reset functionality.
  Administrator Features (YouTube)
- **Recommendation Features:** This video illustrates the intelligent recommendation engine, showing how the system suggests related products to users based on their browsing history and purchase patterns to enhance the shopping experience.
  Recommendation Features (YouTube)
- **Azure Deployment:** This demonstration details the cloud infrastructure setup, covering the deployment process on Azure, the configuration of environment variables, and the monitoring of microservices in a production environment.
  Azure Deployment (YouTube)

11 Source Code Documentation

For a deeper understanding of the project's codebase, initialization procedures, and detailed API references, please refer to the supplementary documentation provided within the repository.

The documentation is structured as follows:

- **Global Project Guide:** Comprehensive information regarding the project initialization, global architecture, and command-line API usage can be found in the main documentation file located at:
  `LINFO2145-2025-2026/project/src/documentation.md`

- **Service-Specific Details:** Detailed technical specifications for each microservice are available in their respective directories at:
  `LINFO2145-2025-2026/project/src/<service-name>/`

12 Conclusion

The completion of this project within the framework of the Cloud Computing course provided a practical opportunity to master the challenges associated with designing and deploying a modern distributed application. By transitioning from a monolithic approach to a microservices architecture, we successfully isolated functional responsibilities (users, products, cart, logging) and ensured superior code maintainability.

The implementation of Docker Swarm as an orchestrator was pivotal in ensuring horizontal scalability and high availability, capabilities that were validated through our load testing. Furthermore, the integration of CouchDB for distributed persistence demonstrates our ability to leverage NoSQL databases in a micro-services environment.

By strictly adhering to the evaluation criteria, including full containerization, state management, and comprehensive API documentation, we have established a solid and robust foundation.

In the second phase, we successfully leveraged this infrastructure to implement an intelligent **Recommendation Engine**. By utilizing CouchDB's Map/Reduce paradigm on the data collected by our Logging Service, we were able to generate meaningful product suggestions, thereby completing the transition from a static catalog to a dynamic, data-driven e-commerce platform.

The successful deployment of this architecture on the **Microsoft Azure** cloud further validates our ability to manage real-world production environments, handling networking, security, and distributed orchestration effectively.